

Semaine 2

HTML5 — Structure & sémantique

Anne Terrier — Développement web, classe passerelle

July 28, 2026

Cours 4h

TP 4h

- Écrire une page HTML5 valide et sémantique
- Structurer le contenu avec les éléments appropriés
- Maîtriser le texte, les liens, les images, les listes et les tableaux
- Créer des formulaires fonctionnels avec les types d'input HTML5
- Prendre en compte les bases de l'accessibilité

1 Qu'est-ce que le HTML ?

En semaine 1, nous avons vu que trois langages coexistent dans le navigateur. Le premier d'entre eux, le **HTML** (*HyperText Markup Language*), est le langage qui décrit la **structure** et le **contenu** d'une page web. Pour créer une page web, il suffit de créer un fichier avec l'extension **.html**.

Le HTML n'est pas un langage de programmation : c'est un **langage de balisage**. On *balise* le contenu à l'aide d'éléments (*balises*) pour indiquer au navigateur ce que chaque morceau représente : un titre, un paragraphe, une image, un lien...

Anatomie d'une balise

La plupart des éléments HTML fonctionnent par **paires** : une balise ouvrante et une balise fermante, qui encadrent un contenu.

`<p> Bonjour tous le monde </p>`

↑ ↑ ↑

Balise ouvrante Contenu Balise fermante

Certaines balises sont **auto-fermantes** : elles n'encadrent aucun contenu et s'écrivent seules, comme `<img>` (une image) ou `<br>` (un retour à la ligne).

Les attributs

Une balise peut porter des **attributs** qui précisent son comportement ou ses propriétés. Un attribut s'écrit dans la balise ouvrante, sous la forme `nom="valeur"`.

```
1 <a href="https://www.hepia.ch">Site de la hepia</a>
```

Extrait 1: Un lien avec l'attribut href

Ici, `href` est l'attribut qui indique la destination du lien.

2 La structure d'un document HTML5

Toute page HTML5 suit le même squelette de base. Le voici, à connaître par cœur :

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Titre de la page</title>
7 </head>
8 <body>
9   <!-- Le contenu visible de la page va ici -->
10 </body>
11 </html>
```

Extrait 2: Squelette minimal d'une page HTML5

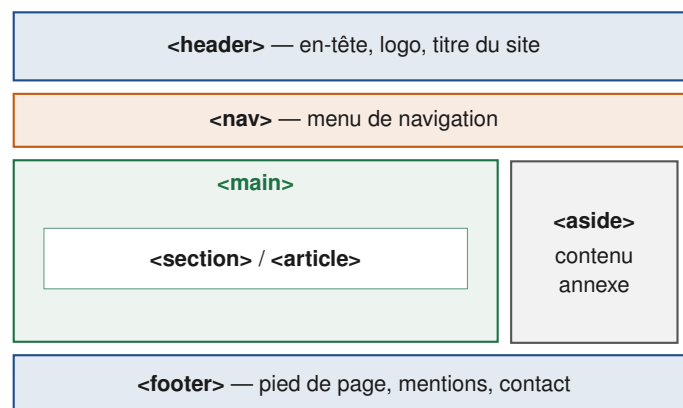
Élément	Rôle
<!DOCTYPE html>	Indique au navigateur qu'il s'agit d'un document HTML5
<html lang="fr">	La racine du document ; lang précise la langue
<head>	Métadonnées invisibles : encodage, titre, feuilles de style. ...
<meta charset>	Définit l'encodage des caractères (toujours UTF-8)
<title>	Le titre affiché dans l'onglet du navigateur
<body>	Tout le contenu visible de la page

Important — head vs body

Tout ce qui doit être **vu** par le visiteur va dans le <body>. Le <head> ne contient que des informations *sur* la page (son titre, son encodage), jamais du contenu affiché.

3 Les éléments sémantiques

Avant HTML5, les pages étaient construites avec des <div> partout. HTML5 a introduit des balises **sémantiques** : elles décrivent le *rôle* de chaque zone de la page, ce qui la rend plus lisible pour les développeurs, les moteurs de recherche et les outils d'accessibilité.



Balise	Utilisation
<code><header></code>	En-tête de la page ou d'une section (titre, logo)
<code><nav></code>	Zone de navigation (menu principal, liens internes)
<code><main></code>	Le contenu principal, unique dans la page
<code><section></code>	Un regroupement thématique de contenu
<code><article></code>	Un contenu autonome (un message du forum, un billet)
<code><aside></code>	Un contenu annexe (barre latérale, encadré)
<code><footer></code>	Pied de page (mentions, contact, copyright)

Retiens la règle simple : un `<article>` a du sens **tout seul**, sorti de son contexte (par exemple un message de forum que l'on pourrait citer ailleurs). Une `<section>` regroupe des contenus liés par un **thème** au sein de la page.

4 Texte, liens et images

Structurer le texte

```

1 <h1>Titre principal (un seul par page)</h1>
2 <h2>Sous-titre de niveau 2</h2>
3 <h3>Sous-titre de niveau 3</h3>
4
5 <p>Un paragraphe de texte. On peut y mettre du
6   <strong>texte important</strong> ou de
7   l'<em>emphase</em>.</p>

```

Extrait 3: Titres et paragraphes

Hiérarchie des titres

Les titres vont de `<h1>` à `<h6>`. Il ne faut en utiliser **qu'un seul** `<h1>` par page, et ne pas sauter de niveau (ne pas passer d'un `<h2>` directement à un `<h4>`). Cette hiérarchie est essentielle pour l'accessibilité et le référencement.

Les liens

Le lien hypertexte, cœur du web, se crée avec la balise `<a>` et son attribut `href` :

```

1 <!-- Lien externe -->
2 <a href="https://www.hepia.ch">Site de la hepia</a>
3
4 <!-- Lien vers une autre page du site -->
5 <a href="contact.html">Nous contacter</a>
6
7 <!-- Lien ouvrant un nouvel onglet -->
8 <a href="https://moodle.org" target="_blank">Moodle</a>
9
10 <!-- Lien interne : vers un element de la meme page -->
11 <a href="#contact">Aller au formulaire</a>

```

Extrait 4: Différents types de liens

Ce dernier exemple, le **lien interne**, mérite qu'on s'y arrête : le `#contact` ne pointe pas vers un fichier, mais vers un **élément de la page même** — celui qui porte l'identifiant `contact`. Pour comprendre comment, il faut découvrir l'attribut `id`.

L'attribut `id` : identifier un élément

L'attribut `id` donne un **nom unique** à un élément de la page. On peut ensuite « pointer » vers cet élément.

```
1 <!-- Le lien pointe vers l'element d'id "contact" -->
2 <a href="#contact">Aller au formulaire</a>
3
4 <!-- ... plus bas dans la page ... -->
5 <section id="contact">
6   <h2>Nous contacter</h2>
7 </section>
```

Extrait 5: Un `id` sert de cible à un lien interne

Cliquer sur le lien fait **défiler** la page jusqu'à la section `contact`. Le `#` dans le `href` signifie « un élément de cette page », suivi de l'`id` visé.

Règle d'or — un `id` est unique

Sur une même page, **deux éléments ne peuvent jamais partager le même `id`**. C'est ce qui garantit qu'un `id` désigne un élément, et un seul. (À la différence de l'attribut `class`, que nous verrons avec le CSS, et qui peut, lui, être partagé.)

L'`id` ne sert pas qu'aux liens internes. C'est un **point d'accroche** vers un élément, réutilisé partout ensuite : il relie déjà un `<label>` à son champ (via `for`), il permettra de cibler un élément en **CSS** (semaine 3), et de le manipuler en **JavaScript** — souviens-toi du tout premier exemple : `document.getElementById('btn')` allait chercher le bouton par son `id`.

Les images

La balise `<img>` est auto-fermante. Elle a deux attributs importants :

```
1 
```

Extrait 6: Insérer une image

Attribut	Rôle
<code>src</code>	Le chemin vers le fichier image (<i>source</i>)
<code>alt</code>	Un texte alternatif décrivant l'image

L'attribut `alt` n'est pas optionnel

Le texte `alt` est lu à voix haute par les lecteurs d'écran pour les personnes malvoyantes, et s'affiche si l'image ne charge pas. Il est **obligatoire** pour une page accessible. Décris ce que montre l'image, pas « image de... ».

5 Listes et tableaux

Les listes

Il existe deux types de listes courantes :

```
1 <!-- Liste à puces (unordered list) -->
2 <ul>
3   <li>HTML</li>
4   <li>CSS</li>
5   <li>JavaScript</li>
6 </ul>
7
8 <!-- Liste numérotée (ordered list) -->
9 <ol>
10  <li>Ouvrir le terminal</li>
11  <li>Taper la commande</li>
12  <li>Valider</li>
13 </ol>
```

Extrait 7: Liste non ordonnée et liste ordonnée

Utilise une liste **ordonnée** () quand l'ordre compte (des étapes à suivre), et une liste **non ordonnée** () quand l'ordre est indifférent (une liste de fonctionnalités).

Les tableaux

Un tableau se construit ligne par ligne (<tr>), avec des cellules d'en-tête (<th>) et des cellules de données (<td>).

```
1 <table>
2   <thead>
3     <tr>
4       <th>Sujet</th>
5       <th>Auteur</th>
6     </tr>
7   </thead>
8   <tbody>
9     <tr>
10      <td>Bienvenue</td>
11      <td>Alice</td>
12    </tr>
13  </tbody>
14 </table>
```

Extrait 8: Un tableau simple

Fusionner des cellules Il arrive qu'une cellule doive s'étendre sur plusieurs colonnes ou plusieurs lignes. On utilise alors les attributs `colspan` (fusion horizontale) et `rowspan` (fusion verticale).

```
1 <table>
2   <tr>
3     <th colspan="2">Coordonnées</th>
4   </tr>
5   <tr>
6     <td>Email</td>
```

```

7     <td>alice@exemple.ch</td>
8   </tr>
9 </table>

```

Extrait 9: Fusion de cellules avec colspan

Ici, la cellule d'en-tête « Coordonnées » occupe la largeur de **deux** colonnes grâce à `colspan="2"`. L'attribut `rowspan="2"` produirait le même effet sur la hauteur, en fusionnant deux lignes.

Les tableaux, c'est pour des données

Un tableau HTML sert à présenter des **données tabulaires** (un planning, des résultats). Il ne faut **jamais** l'utiliser pour mettre en page une interface : c'est le rôle du CSS, que nous verrons en semaine 3.

6 Les formulaires

Les formulaires permettent à l'utilisateur de **saisir** des informations : c'est ce qui rendra le MiniForum interactif. La balise conteneur est `<form>`.

Le champ de saisie : `<input>`

La balise `<input>` change de comportement selon son attribut `type`. HTML5 en propose de nombreux, avec validation automatique :

type	Usage
text	Texte libre sur une ligne
email	Adresse e-mail (validée automatiquement)
password	Mot de passe (caractères masqués)
number	Valeur numérique
checkbox	Case à cocher
radio	Bouton radio (choix unique)

Cases à cocher et boutons radio

Ces deux types méritent un mot, car leur usage est un peu particulier. Une **case à cocher** (checkbox) permet plusieurs sélections indépendantes ; un **bouton radio** (radio) impose un choix **unique** parmi plusieurs.

```

1 <!-- Cases à cocher : plusieurs choix possibles -->
2 <input type="checkbox" id="news" name="news">
3 <label for="news">Recevoir la newsletter</label>
4
5 <!-- Boutons radio : un seul choix (meme name !) -->
6 <input type="radio" id="pub" name="visibilite" value="public">
7 <label for="pub">Public</label>
8 <input type="radio" id="priv" name="visibilite" value="prive">
9 <label for="priv">Prive</label>

```

Extrait 10: Cases à cocher et boutons radio

La règle des boutons radio

Pour que des boutons radio fonctionnent comme un groupe (un seul sélectionnable à la fois), ils doivent **partager le même attribut** `name`. C'est cet attribut commun qui les relie ; l'attribut `value` distingue, lui, l'option choisie.

Un formulaire complet

```
1 <form>
2   <label for="pseudo">Votre pseudo :</label>
3   <input type="text" id="pseudo" name="pseudo"
4     placeholder="ex : alice" required>
5
6   <label for="message">Votre message :</label>
7   <textarea id="message" name="message"
8     maxLength="500" required></textarea>
9
10  <label for="categorie">Categorie :</label>
11  <select id="categorie" name="categorie">
12    <option value="general">General</option>
13    <option value="aide">Aide</option>
14  </select>
15
16  <button type="submit">Envoyer</button>
17 </form>
```

Extrait 11: Formulaire d'ajout de message

Élément	Rôle
<label>	L'étiquette décrivant un champ
<input>	Un champ de saisie (selon son type)
<textarea>	Une zone de texte multi-lignes
<select>	Une liste déroulante de choix
<button>	Un bouton (ici, pour envoyer le formulaire)

Le lien label / input

L'attribut `for` du `<label>` doit correspondre à l'id de l'`<input>` associé. Ce lien permet de cliquer sur l'étiquette pour activer le champ, et il est indispensable pour l'accessibilité.

Rendre un formulaire plus fiable

On peut guider l'utilisateur et éviter les saisies erronées grâce à quelques attributs, sans écrire une seule ligne de JavaScript.

Attribut	Effet
placeholder	Affiche un texte d'exemple grisé dans le champ vide
required	Rend le champ obligatoire : le formulaire refuse de partir s'il est vide
maxlength	Limite le nombre de caractères saisissables
min / max	Bornent une valeur numérique (avec <code>type="number"</code>)

Le navigateur vérifie **lui-même** certains champs avant l'envoi : un `type="email"` refuse une adresse mal formée, un champ `required` vide bloque l'envoi. Cette **validation HTML5** est immédiate, sans JavaScript, et améliore nettement l'expérience utilisateur. Nous la compléterons plus tard côté serveur, car une validation côté navigateur ne suffit jamais à elle seule pour la sécurité.

Tu as sans doute remarqué que ce formulaire ne va, pour l'instant, **nulle part** : cliquer sur « Envoyer » ne fait rien de visible. C'est normal. L'envoi réel des données vers un **serveur** (via les attributs `action` et `method` de la balise `<form>`) sera vu en **phase 3**, quand le MiniForum aura un serveur Node.js pour les recevoir.

7 Accessibilité : les bons réflexes

L'**accessibilité** consiste à rendre les pages utilisables par tous, y compris les personnes en situation de handicap (malvoyance, navigation au clavier...). Dès la semaine 2, quelques réflexes simples font une grande différence :

- Toujours renseigner l'attribut `alt` des images.
- Respecter la hiérarchie des titres (`<h1>` puis `<h2>`... sans sauter de niveau).
- Associer chaque `<input>` à un `<label>` via `for` / `id`.
- Utiliser les balises sémantiques (`<nav>`, `<main>`...) plutôt que des `<div>` génériques.
- Préciser la langue de la page avec `<html lang="fr">`.

Le service en ligne **validator.w3.org** vérifie qu'une page HTML est correctement écrite (balises bien fermées, structure valide). Prends l'habitude de valider ton code : un HTML valide est la base d'un site fiable.

8 Lexique de la semaine

Terme	Définition
Balise (<i>tag</i>)	Élément HTML qui décrit un morceau de contenu
Attribut	Information complémentaire placée dans une balise (<code>nom="valeur"</code>)
Sémantique	Choix d'une balise selon le <i>sens</i> du contenu, pas son apparence
Élément auto-fermant	Balise sans contenu ni fermeture (<code></code> , <code>
</code>)
<code>id</code>	Identifiant unique d'un élément ; cible d'un lien interne, d'un label, du CSS et du JS
<code><head></code>	Zone des métadonnées, invisible pour le visiteur
<code><body></code>	Zone du contenu visible de la page
Accessibilité	Ensemble des pratiques rendant une page utilisable par tous
<code>alt</code>	Texte alternatif décrivant une image
Formulaire	Ensemble de champs permettant à l'utilisateur de saisir des données
<code>required</code>	Attribut rendant un champ obligatoire avant l'envoi
Validation HTML5	Vérification automatique des champs par le navigateur, sans JavaScript

Pour aller plus loin

- Un **quiz d'auto-évaluation** sur cette semaine est disponible sur Moodle.
- La **fiche TP** (squelette HTML du MiniForum) est distribuée séparément : *S02_TP.pdf*.

Résumé de la semaine

- Le HTML est un langage de **balisage** qui décrit la structure du contenu
- Toute page suit le même squelette : `<!DOCTYPE>`, `<head>`, `<body>`
- Les balises **sémantiques** (`<header>`, `<nav>`, `<main>`, `<article>`, `<footer>`) décrivent le rôle de chaque zone
- On structure le texte avec les titres `<h1>...<h6>`, les liens `<a>`, les images `<img>`
- Les **formulaires** (`<form>`, `<input>`, `<label>`) permettent la saisie de données
- L'**accessibilité** se construit dès le HTML : `alt`, `label`, hiérarchie des titres

Pour la semaine 3

- Avoir terminé le squelette HTML du MiniForum (en-tête, liste de sujets, formulaire)
- Vérifier que sa page passe le validateur **validator.w3.org** sans erreur
- La semaine 3 introduira le **CSS** pour donner du style à cette structure